Lab 6. Using Docker with Jenkins Pipelines

By the end of this lab exercise, you should be able to:

- Prepare the Jenkins environment to build with a Docker agent
- Refactor a Jenkinsfile with a Docker-based agent

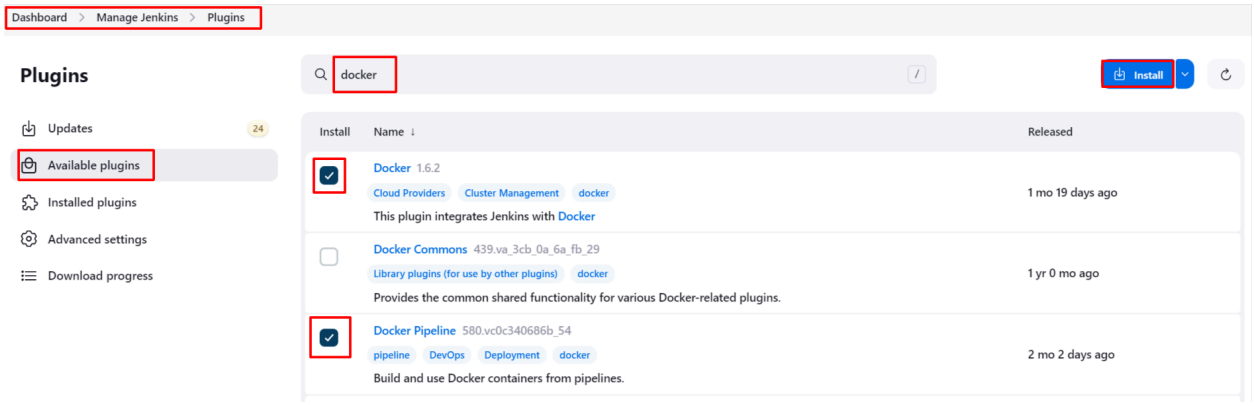
Steps:

- Read *"Using Docker with Pipeline"*
- Refactor the Jenkinsfile for the `worker` application

Install Docker Plugins

Install the following plugins:

- Docker Plugin
- Docker Pipeline



Refer to the screenshot above to choose the correct plugins and proceed with the installation.

Refactoring the Worker Pipeline to Build with Docker Agent

Create a new branch to make changes.

```
git checkout -b feature/dockerbuild
```

Now update the Jenkinsfile for the **`worker`** app:

- Remove the **`tools`** section
- Change the agent configuration from **`any`** to **`docker`**
- Provide the Docker agent configurations with
  - image: **`maven:3.9.8-sapmachine-21`**
  - args: **`'-v $HOME/.m2:/root/.m2'`**

Your **`worker/Jenkinsfile`** should contain the following:

```
pipeline {

    agent{
       docker{
         image 'maven:3.9.8-sapmachine-21'
         args '-v $HOME/.m2:/root/.m2'
       }

    }

    stages{
        stage('build'){
            steps{
                echo 'building worker app'
```

```
                dir('worker'){
                  sh 'mvn compile'
                }
            }
        }
        stage('test'){
            steps{
                echo 'running unit tests on worker app'
                dir('worker'){
                  sh 'mvn clean test'
                }
            }
        }
        stage('package'){
            steps{
                echo 'packaging worker app into a jarfile'
                dir('worker'){
                  sh 'mvn package -DskipTests'
                  archiveArtifacts artifacts: '**/target/*.jar',
fingerprint:  true
                }
            }
        }
    }
    post{
        always{
            echo 'the job is complete'
        }

    }

}
```

Once you have modified the file, commit the changes:

```
git commit -am "add docker based agent"
```

```
git push origin feature/dockerbuild
```

Jenkins launches the pipeline, this time with Docker as an agent.

Validation

While the job is running, log into the Docker DIND Host and watch for the container which gets

launched when the pipeline starts and stays till the end of the build. Switch into **devops-repo/setup**:

```
$ docker compose exec docker sh
$ watch docker ps

...
...
[observe the container running while the pipeline is in progress]
...
..
[^c followed by ^d to stop the watch command and exit from the
container]
```

You should see containers being launched periodically by Jenkins. You can also observe the console logs for the pipeline to see a container being launched, then stopped, and finally removed as part of the build process. You can always view the pipeline console output by clicking the build number and clicking **Console Output** on the menu on the right.

Once you've done this, remember to merge the changes into the master branch via a pull request and then delete the feature branch from remote (GitHub) and locally.

Summary

In this lab, we built a simple Docker pipeline to ensure that Docker was working on Jenkins. We refactored our **worker** application Jenkinsfile so that the pipeline would run on Docker.